

02 zip 推导式与函数综合

专项目标：让数据结构进入函数，让函数返回可继续处理的数据结构，再用 `zip`、`enumerate`、推导式、`lambda` 完成排序、筛选、统计。

总模型

数据结构负责装数据
函数负责处理数据

外部数据

- 参数传入
- 函数内部遍历和统计
- `return` 返回结果
- 调用者继续使用

考试里最重要的判断：

函数接收什么？
函数内部每轮循环拿到什么？
函数返回什么？
调用者拿到返回值后还能不能继续用？

函数基础【必须闭卷会写】

1. 它解决什么问题

函数把一段过程封装起来，让程序可复用、可测试。实验报告里的角谷猜想、最大公约数、登录校验、回文判断，都适合拆成函数。

2. 最小语法

```
def add(a, b):  
    return a + b  
  
result = add(3, 5)  
print(result)
```

3. 执行过程

```
调用 add(3, 5)
→ a 接收 3, b 接收 5
→ return a + b
→ 调用处拿到 8
```

4. print 与 return

```
def f1(a, b):
    print(a + b)

def f2(a, b):
    return a + b

x = f1(3, 5)
y = f2(3, 5)

print(x)  # None
print(y)  # 8
```

记忆：

```
print 是给人看
return 是给程序继续用
```

5. 返回结构化结果

```
def analyze_students(students):
    total = 0
    passed = 0
    top = students[0]

    for student in students:
        score = student["score"]
        total += score

        if score >= 60:
            passed += 1

        if score > top["score"]:
            top = student
```

```
average = total / len(students)

return {
    "top": top,
    "average": average,
    "passed": passed,
}
```

为什么返回字典而不是只打印？

返回字典后，调用者可以继续取 `result["top"]`、`result["average"]`。
只打印就没法继续计算。

参数类型【必须闭卷会写】

类型	写法	说明
形参	<code>def f(name):</code>	函数定义处的变量
实参	<code>f("Tom")</code>	调用时传入的值
位置参数	<code>f("Tom", 80)</code>	按顺序匹配
关键字参数	<code>f(score=80, name="Tom")</code>	按名字匹配
默认参数	<code>def f(name, score=0):</code>	不传就用默认值

默认参数必须放普通参数后面。

*args 与 **kwargs【必须会读会改】

1. 定义时收包

```
def total(*args):
    print(args)
    result = 0
    for num in args:
        result += num
    return result

print(total(1, 2, 3))
```

args 是元组。

```
def show_info(**kwargs):  
    print(kwargs)  
    for key, value in kwargs.items():  
        print(key, value)  
  
show_info(name="Tom", score=80)
```

kwargs 是字典。

2. 调用时解包

```
def add(a, b):  
    return a + b  
  
numbers = (3, 4)  
print(add(*numbers))
```

```
def show_student(name, score):  
    print(name, score)  
  
student = {"name": "Tom", "score": 80}  
show_student(**student)
```

重点：

定义时 * / ** 是收起来
调用时 * / ** 是拆开来
使用 **student 时，字典键必须和函数参数名一致

错误：

```
def show_student(name, score):  
    print(name, score)  
  
student = {"username": "Tom", "score": 80}  
show_student(**student) # TypeError, 因为 username 对不上 name
```

zip 【必须闭卷会写】

1. 它解决什么问题

`zip` 把多个相关列表按位置配对。姓名列表和成绩列表不能单独排序，否则关系会乱。

2. 最小语法

```
names = ["Tom", "Jerry", "Alice"]
scores = [80, 92, 55]

for name, score in zip(names, scores):
    print(name, score)
```

3. 执行过程

```
zip(names, scores)
→ ("Tom", 80)
→ ("Jerry", 92)
→ ("Alice", 55)
```

4. 普通循环写法

```
for i in range(len(names)):
    name = names[i]
    score = scores[i]
    print(name, score)
```

5. 常见用法

```
names = ["Tom", "Jerry", "Alice"]
scores = [80, 92, 55]
classes = ["A", "B", "A"]

print(list(zip(names, scores)))
print(list(zip(names, scores, classes)))
print(dict(zip(names, scores)))
```

`zip` 以最短序列为准：

```
print(list(zip(["A", "B"], [1, 2, 3])))
# [('A', 1), ('B', 2)]
```

反向拆分：

```
pairs = [("Tom", 80), ("Jerry", 92)]
names2, scores2 = zip(*pairs)
print(list(names2))
print(list(scores2))
```

6. zip 与字典列表

```
names = ["Tom", "Jerry", "Alice"]
scores = [80, 92, 55]

students = []
for name, score in zip(names, scores):
    students.append({"name": name, "score": score})

print(students)
```

推导式写法：

```
students = [
    {"name": name, "score": score}
    for name, score in zip(names, scores)
]
```

7. zip 与 sorted

错误：只排序分数。

```
names = ["Tom", "Jerry", "Alice"]
scores = [80, 92, 55]
scores.sort()
print(list(zip(names, scores))) # 对应关系错了
```

正确：先整体打包再排序。

```
students = list(zip(names, scores))
students_sorted = sorted(students, key=lambda item: item[1], reverse=True)
print(students_sorted)
```

enumerate 【必须闭卷会写】

1. 它解决什么问题

遍历列表时同时拿到编号和值。

2. 最小语法

```
names = ["Tom", "Jerry"]

for index, name in enumerate(names, start=1):
    print(index, name)
```

3. 与 range(len(...)) 对比

```
for i in range(len(names)):
    print(i + 1, names[i])
```

`enumerate` 更直接：每轮拿到（索引，元素）。

三种推导式【必须会读会改】

推导式阅读顺序：

```
先看 for
再看 if
最后看最前面的结果表达式
```

1. 列表推导式

普通循环：

```
result = []
for score in [80, 55, 92]:
    if score >= 60:
        result.append(score)
print(result)
```

推导式：

```
result = [score for score in [80, 55, 92] if score >= 60]
print(result)
```

2. 字典推导式

```
names = ["Tom", "Jerry"]
scores = [80, 92]

score_dict = {name: score for name, score in zip(names, scores)}
print(score_dict)
```

3. 集合推导式

```
nums = [1, 2, 2, 3]
unique_squares = {num ** 2 for num in nums}
print(unique_squares)
```

4. 生成器表达式【必须会读会改】

常和 `sum`、`any`、`all`、`max`、`min` 搭配。

```
scores = [80, 55, 92]
passed_count = sum(1 for score in scores if score >= 60)
print(passed_count)
```

5. 判断错变量案例

错误：

```
n = 5
result = [i ** 2 for i in range(1, n + 1) if n % 2 == 0]
```

正确：

```
n = 5
result = [i ** 2 for i in range(1, n + 1) if i % 2 == 0]
```

解释：循环变量是 `i`，筛选条件应判断当前元素。

lambda、sorted、max、min【必须会读会改】

1. lambda 本质

普通函数：

```
def get_score(student):  
    return student["score"]
```

等价 lambda：

```
lambda student: student["score"]
```

2. 字典列表排序

```
students = [  
    {"name": "Tom", "score": 80},  
    {"name": "Jerry", "score": 92},  
    {"name": "Alice", "score": 55},  
]  
  
sorted_students = sorted(students, key=lambda student: student["score"],  
reverse=True)  
print(sorted_students)
```

3. max 返回原始元素

```
top = max(students, key=lambda student: student["score"])  
print(top)
```

重点：

key 返回比较依据
max 返回原始元素
top 是完整学生字典，不是单独的分

4. 多条件排序【必须会读会改】

```
students = [  
    {"name": "Tom", "score": 80},  
    {"name": "Amy", "score": 80},  
    {"name": "Jerry", "score": 92},  
]  
  
result = sorted(students, key=lambda student: (-student["score"],
```

```
student["name"]))  
print(result)
```

含义：成绩降序，成绩相同按姓名升序。

any 和 all 【必须会读会改】

```
scores = [80, 55, 92]  
  
print(any(score >= 60 for score in scores)) # 至少一个及格  
print(all(score >= 60 for score in scores)) # 全部及格
```

空序列结果了解即可。

可变对象与函数 【必须会读会改】

1. append 会修改原列表

```
def add_score(scores, score):  
    scores.append(score)  
  
data = [80, 90]  
add_score(data, 70)  
print(data)
```

2. 重新赋值不会清空外部列表

```
def reset(scores):  
    scores = []  
  
data = [80, 90]  
reset(data)  
print(data) # [80, 90]
```

3. clear() 会修改原列表

```
def reset(scores):  
    scores.clear()
```

```
data = [80, 90]
reset(data)
print(data) # []
```

4. 默认可变参数陷阱

错误：

```
def add_student(name, students=[]):
    students.append(name)
    return students
```

正确：

```
def add_student(name, students=None):
    if students is None:
        students = []
    students.append(name)
    return students
```

作用域【必须会读会改】

Python 找变量按 LEGB：

Local 当前函数
Enclosing 外层函数
Global 全局
Built-in 内置

`global` 修改全局变量：

```
count = 0

def add_one():
    global count
    count += 1
```

`nonlocal` 修改外层函数变量：

```
def outer():
    count = 0

    def inner():
        nonlocal count
        count += 1
        return count

    return inner
```

综合题优先使用参数和 `return`，不要滥用 `global`。

递归最小范围【必须会读会改】

递归求和

```
def total(n):
    if n == 1:
        return 1
    return n + total(n - 1)

print(total(5))
```

阶乘

```
def factorial(n):
    if n == 1:
        return 1
    return n * factorial(n - 1)
```

倒计时

```
def count_down(n):
    if n == 0:
        print("结束")
        return
    print(n)
    count_down(n - 1)
```

递归必须有：

终止条件
参数逐步接近终止条件

三条完整数据流水线

流水线 A：成绩字符串

字符串
→ split
→ int
→ 列表
→ 函数统计
→ 字典结果

```
def analyze_scores(text):  
    scores = []  
    for part in text.split():  
        scores.append(int(part))  
  
    return {  
        "highest": max(scores),  
        "lowest": min(scores),  
        "average": sum(scores) / len(scores),  
    }  
  
print(analyze_scores("80 90 70"))
```

流水线 B：姓名与成绩

姓名列表 + 成绩列表
→ zip
→ 字典列表
→ lambda 排序
→ 函数返回

```
def build_students(names, scores):  
    students = []  
    for name, score in zip(names, scores):  
        students.append({"name": name, "score": score})  
    return students
```

```
def sort_students(students):  
    return sorted(students, key=lambda student: student["score"],  
reverse=True)
```

流水线 C：学生文本

```
"Tom,80"  
→ parse_student()  
→ 学生字典  
→ 学生列表  
→ analyze_students()  
→ format_student()  
→ join 输出
```

```
def parse_student(line):  
    name, score_text = line.strip().split(",")  
    return {"name": name, "score": int(score_text)}  
  
def format_student(student):  
    return f'{student["name"]}:{student["score"]}'  
  
student = parse_student("Tom,80")  
print(format_student(student))
```

函数与数据结构关系图

```
list[str] 原始文本  
    ↓ parse  
list[dict] 学生列表  
    ↓ analyze_students(students)  
dict 统计结果  
    ↓ format  
str 输出文本
```

参数和返回值速查

目标	推荐返回
判断是否通过	bool

目标	推荐返回
计算一个结果	int / float / str
返回多个固定值	tuple
返回一批同类数据	list
返回带名字的统计结果	dict

zip/推导式/lambda 速查

```
dict(zip(names, scores))
list(enumerate(names, start=1))
[x for x in nums if x > 0]
{name: score for name, score in zip(names, scores)}
{x for x in nums}
sorted(students, key=lambda s: s["score"])
```

12 个易错点

1. 函数忘记 `return`，调用处拿到 `None`。
2. 把 `print()` 当成返回值。
3. 默认参数使用空列表。
4. `*args` 是元组，不是列表。
5. `**kwargs` 是字典。
6. 调用时 `**student` 的键必须匹配参数名。
7. `zip` 以最短序列为准。
8. 关联列表不能单独排序。
9. 推导式条件判断错变量。
10. `lambda` 只能写表达式，不能写多行语句。
11. `max(..., key=...)` 返回原元素。
12. 综合题优先参数和返回值，不滥用 `global`。

10 个口头自测问题

1. `print` 和 `return` 的区别是什么？
2. 函数返回多个值，本质是什么结构？
3. `*args` 和 `**kwargs` 分别得到什么？
4. 定义时 `*` 和调用时 `*` 含义一样吗？
5. `zip` 为什么适合处理姓名和成绩？
6. `enumerate(start=1)` 每轮拿到什么？

7. 推导式阅读顺序是什么？
8. `lambda student: student["score"]` 返回什么？
9. `any` 和 `all` 的区别是什么？
10. 为什么默认参数不要写 `[]` ？

6 个最小闭卷函数题

题 1

写函数 `parse_scores(text)`，把 `"80 90 75"` 转成整数列表并返回。

题 2

写函数 `get_min_max(scores)`，返回最低分和最高分。

题 3

写函数 `build_score_dict(names, scores)`，返回姓名到成绩的字典。

题 4

写函数 `filter_passed(students)`，返回及格学生列表。

题 5

写函数 `sort_by_score(students)`，按成绩降序返回新列表。

题 6

写递归函数 `recursive_sum(n)`，返回 `1 + 2 + ... + n`。

答案区

```
def parse_scores(text):
    scores = []
    for part in text.split():
        scores.append(int(part))
    return scores

def get_min_max(scores):
    return min(scores), max(scores)
```



```
def build_score_dict(names, scores):  
    return dict(zip(names, scores))  
  
def filter_passed(students):  
    result = []  
    for student in students:  
        if student["score"] >= 60:  
            result.append(student)  
    return result  
  
def sort_by_score(students):  
    return sorted(students, key=lambda student: student["score"],  
reverse=True)  
  
def recursive_sum(n):  
    if n == 1:  
        return 1  
    return n + recursive_sum(n - 1)
```

[返回字符串与数据结构基础](#)

[进入闭卷训练](#)